

GESTION DINÁMICA DE MEMORIA CON ARRAYS.

Hasta el momento, en los programas que manejan matrices se han utilizado arrays de tamaño ya definido durante su declaración (`A[5][3]`), de tal forma que durante la compilación del programa se reserva, de forma permanente, la memoria necesaria para dichos arrays. Conviene recordar, que la declaración de un array en C++ exige especificar todas sus dimensiones, excepto la primera, con un valor constante, ya que si se utiliza una variable o una expresión que contiene variables se producirá un error durante la compilación. En la práctica, lo más habitual es especificar todas las dimensiones del array con valores constantes. Por ejemplo: `double A[4][2], B[11];` .

Por el contrario, hay situaciones en las que no conocemos de antemano la memoria necesaria hasta el momento de la ejecución del programa, es decir las dimensiones del array. Por ejemplo, podemos estar interesados en pedir al usuario que introduzca por teclado el número de filas y columnas de una matriz, y reservar después la memoria necesaria para almacenar dicha matriz. En esta situación la reserva de memoria se hace dinámicamente durante la ejecución del programa en función de los datos recogidos del usuario. Para ello, se utilizan técnicas de asignación dinámica de memoria, que permiten reservar y liberar memoria durante la ejecución del programa de acuerdo a las necesidades.

Para llevar a cabo la reserva dinámica de memoria en C++ se utilizan dos operadores: **new** y **delete**:

Operador new:

Se utiliza para reservar o asignar un bloque de memoria libre donde almacenar nuestro array. Este operador devuelve un puntero, es decir la dirección del bloque de memoria asignado dinámicamente para almacenar el array.

Operador delete:

Cuando se ha terminado de utilizar el array dinámico, es decir un bloque de memoria previamente asignado con el operador new, podemos estar interesados en liberar dicha memoria para que este disponible para otros usos. Para ello se utiliza el operador delete.

A continuación, vamos a ver en detalle el uso de los operadores new y delete con arrays unidimensionales (vectores) y bidimensionales (matrices).

Arrays unidimensionales (vectores):

Supongamos que queremos reservar memoria para un vector A de n elementos enteros, donde n se le pide al usuario por teclado durante la ejecución del programa.

En primer lugar, hay que declarar A como un puntero: → `int *A;`

Es conveniente recordar que el nombre de un array unidimensional, A, es un puntero que apunta a la dirección de memoria donde está almacenado el primer elemento de dicho vector.

La sentencia clave es la que se muestra a continuación, que tiene la siguiente estructura:

```
puntero = new tipo[dimension] → A= new int [n] ;
```

que reserva memoria para un vector A de n elementos de tipo entero.

(*) Se pueden combinar las dos sentencias anteriores en una sólo línea:

```
int *A= new int [n] ;
```

Así mismo, para liberar la memoria previamente reservada se utiliza la siguiente sentencia:

delete [] A; donde en este caso no hace falta especificar la dimensión n del vector.

```
...
int *A;
int n;
cout<< "Introduce el valor de n: ";
cin >> n;
A= new int [n] ; // reserva dinámica de memoria
// rellenamos el vector
for(int i=0; i < n; i++) {
    A[i] = (i*i+1);
    cout << A[i] << endl;
}
delete [] A; // se libera la memoria ocupada por A
...
```

Arrays bidimensionales (matrices):

En este caso, la reserva dinámica de memoria para una matriz es un poco más elaborada, ya que C++ considera que **un array bidimensional es en realidad un array de punteros a los arrays que forman sus filas**. Por lo tanto, es necesario utilizar en este caso un **puntero doble ó puntero a puntero, (**A)**, que contiene la dirección del primer puntero del array de punteros a cada una de las filas del array bidimensional.

Por ejemplo, si se ha definido una matriz A[5][3] de tipo double, el nombre de dicha matriz, A, es un puntero constante que apunta a la primera fila de A → A[0], y A+1 es un puntero que apunta a la segunda fila de A → A[1]. A su vez, A[0] es un puntero que apunta al primer elemento de la fila 0 que es el A[0][0], y A[1] es un puntero que apunta al primer elemento de la fila 1 que es el A[1][0], como se resume en la tabla adjunta.

Puntero a puntero fila	Puntero a fila	ARRAY BIDIMENSIONAL double A[5][3]			
A →	A[0] →	A[0][0]	A[0][1]	A[0][2]	
A+1 →	A[1] →	A[1][0]	A[1][1]	A[1][2]	
A+2 →	A[2] →	A[2][0]	A[2][1]	A[2][2]	
A+3 →	A[3] →	A[3][0]	A[3][1]	A[3][2]	
A+4 →	A[4] →	A[4][0]	A[4][1]	A[4][2]	

A es un puntero que apunta a un array de 5 punteros (A[0], A[1], A[2], A[3], A[4]).

A[0] es un puntero que apunta al primer elemento de un array de tres elementos (A[0][0], A[0][1], A[0][2]).

A[1] es un puntero que apunta al primer elemento de un array de tres elementos (A[1][0], A[1][1], A[1][2]).

A[2] es un puntero que apunta al primer elemento de un array de tres elementos (A[2][0], A[2][1], A[2][2]).

A[3] es un puntero que apunta al primer elemento de un array de tres elementos (A[3][0], A[3][1], A[3][2]).

A[4] es un puntero que apunta al primer elemento de un array de tres elementos (A[4][0], A[4][1], A[4][2]).

Supongamos que queremos reservar memoria de forma dinámica para una matriz A de tipo double de *fa* filas y *ca* columnas, donde *fa* y *ca* se le piden al usuario por teclado durante la ejecución del programa.

En primer lugar, hay que declarar A como un **puntero a puntero**: → **double **A;**

La utilización de un puntero doble (****A**) implica que en este caso la reserva dinámica (y la posterior liberación) de memoria se lleva a cabo en dos etapas:

Etapla (a): se reserva memoria para el array de punteros que apunta a las filas de A:

puntero = new tipo[dimension] → **A=new double* [fa];**

Conviene insistir que en este caso el tipo es (double *), es decir puntero a double, ya que estamos reservando memoria para un array de *fa* punteros que apuntan a las *fa* filas de A.

Etapla (b) Se reserva memoria para cada uno de los *ca* elementos que forman cada una de las *fa* filas de la matriz A:

for(int i=0;i < fa; i++)
puntero = new tipo[dimension] → { A[i]=new double [ca]; }

Observar, que dichas filas están apuntadas por los punteros A[i], y que en este caso el tipo es (double), ya que estamos reservando memoria para los elementos de tipo double que forman la matriz A de *fa* filas por *ca* columnas.

Conviene insistir en que en el ejemplo anterior:

A es un puntero a un array de *fa* punteros a datos de tipo double.

A[i] es un puntero a un array de *ca* datos de tipo double.

Para liberar la memoria hay que proceder a la inversa, también en dos etapas, como se muestra a continuación:

```
for (int i=0; i < fa; i++) {
    delete[] A[i];
}
delete [] A;
```

```
...
double **A;
int fa, ca;
cout << "Introduce numero de filas: ";
cin >> fa;
cout << endl;
cout<< "Introduce numero de columnas: ";
cin >> ca;
// Reserva dinamica de memoria.
// etapa (a). Se reserva memoria para el array de punteros que
// apuntan a las filas de A
A=new double* [fa]; //etapa (a)
// etapa (b)
//Se reserva memoria para cada uno de los ca elementos que forman
// cada una de las fa filas de la matriz A.
for(int i=0;i < fa; i++)
    { A[i]=new double[ca];} // etapa (b)

// rellenamos la matriz
for(int i=0;i < fa; i++){
    for(int j=0;j < ca; j++){
        A[i][j]=2*i+3*j;
    }
}
// se libera la memoria ocupada por A en dos etapas.
for (int i=0; i < fa; i++) {
    delete[] A[i];
}
delete [] A;
...
```